Experiment 5: Greedy Best First Search

Aim:

To implement Greedy Best First Search that always chooses the node that appears closest to the goal according to the heuristic.

Algorithm:

1. Priority queue ordered solely by heuristic h(n).
2. Always expand the node that looks nearest to the goal.
3. Does not consider the actual path cost from start.
4. Fast but not guaranteed to be optimal.

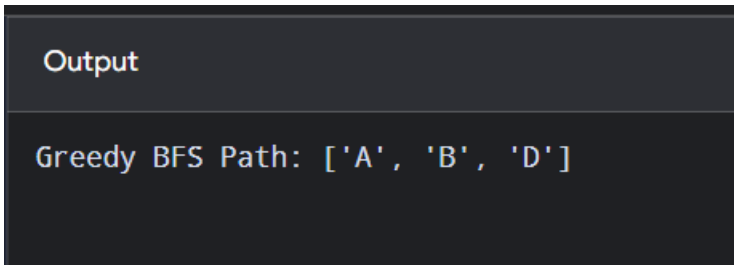
Code:

```
import heapq

def greedy_bfs(graph, start, goal, h):
    pq = [(h[start], start, [start])]
    visited = set()
    while pq:
        _, node, path = heapq.heappop(pq)
        if node in visited: continue
        visited.add(node)
        if node == goal:
            return path
        for n in graph.get(node, []):
            heapq.heappush(pq, (h[n], n, path + [n]))
    return None

graph = {'A':['B','C'], 'B':['D'], 'C':['D'], 'D':[]}
h = {'A':4, 'B':2, 'C':3, 'D':0}
print("Greedy BFS Path:", greedy_bfs(graph, 'A', 'D', h))
```

Sample Output:



Result:

Greedy Best First Search selected the locally best node at each step using only the heuristic and successfully reached the goal.